How to Setup and Run 'Computer Vision Examples for Unity'

'Computer Vision Examples' is the next step in the evolution of depth sensor examples. Instead of a depth sensor this asset uses a plain web camera as input, and AI models to provide depth estimation, body tracking, etc.

1. Create a new Unity project (Unity 6.0 or later, as required by the 'Inference Engine' package).
2. Open 'Window / Package Manager' in Unity editor, go to Unity Registry and install the 'Inference Engine'-package. This is the Unity package for AI model inference.
3. Open 'File / Build settings' and make sure that Windows, macOS or Linux is the currently active platform.
4. Open and run a demo scene that contains body tracking, from a subfolder of the 'ComputerVisionExamples/DemoScenes'-folder (e.g. AvatarDemo1-scene or OverlayDemo2-scene). Stand at some distance, in front of the camera, to calibrate. This is required only once, to estimate the camera intrinsic parameters.
5. Open and run a demo scene of your choice from a subfolder of the 'ComputerVisionExamples/DemoScenes'-folder. Short descriptions of all demo-scenes are available in the online documentation.

Limitations

1. Because of the intensive GPU utilization, please avoid using the CVE-package on mobile platforms. The performance is not good enough, and the device may overheat.
2. Please don't use the CVE-package on WebGL platform.

Short Descriptions of the Available Demo Scenes

Short descriptions of the available demo scenes can be found in the online documentation.

The KinectManager-component

All CV-related scenes require a KinectManager-component. In all demo scenes it is in the KinectController-game object. This is the most basic component that manages the streams and data. Its public API is utilized by all other CV-related components. You can also utilize the public API of KinectManager in your scripts.

Here are the public settings of the KinectManager-component:

Setting:	*Description:*
Get Depth Frames	Whether to get depth frames from the camera(s).
Get Color Frames	Whether to get color frames from the camera(s).
Get Infrared Frames	Whether to get infrared frames from the camera(s), if available.
Get Pose Frames	Whether to get pose frames from the camera(s).
Get Body Frames	Whether to get body frames and/or body-index frames from the body tracker.
Sync Depth and Color	Whether to synchronize the depth and color frames.

Sync Body and Depth	Whether to synchronize the body, body-index and depth frames.
Min User Distance	Minimum distance to user, in order to be considered for body processing. Value of 0 means no minimum distance limitation.
Max User Distance	Maximum distance to user, in order to be considered for body processing. Value of 0 means no maximum distance limitation.
Max Left Right Distance	Maximum left or right distance to user, in order to be considered for body processing. Value of 0 means no left/right distance limitation.
Max Tracked Users	Maximum number of users, who may be tracked simultaneously. Value of 0 means no limitation.
Show Allowed Users Only	Whether to display only the users within the allowed distances, or all users.
User Detection Order	How to assign users to player indices - by order of appearance, distance or left-to-right.
Ignore Inferred Joints	Whether to ignore the inferred body joints or consider them as tracked.
Ignore Z-Coordinates	Whether to ignore the Z-coordinates of the joints (e.g. in 2D-scenes) or not.
Bone Orientation Constraints	Whether to apply the bone orientation constraints.
Player Calibration Pose	Calibration pose required, to start tracking the respective user.
User Manager	User manager, used to track the users. KM creates one, if not set.
Gesture Manager	Gesture manager, used to detect user gestures. KM creates one, if not set.
Display Images	List of images to display on the screen.
Display Image Width	Single image width, as percent of the screen width.
Status Info Text	UI-Text to display status messages.

The CVE-package utilizes an abstract sensor called 'Universal camera' that can be configured to work with different devices and streams. In the CVE-package it uses web cameras (or videos) as input devices, and creates color, depth, and body tracking streams directly, or by running AI model inferences. The utilized stream providers can be set up in the 'UniCamConfig'-asset in the package.

In all demo scenes the UniCam sensor interface resides on a child object of the KinectController-game object. You can change the sensor-interface settings, according to your needs. Please note, the Transform-component of each sensor-interface object represents the camera's position and rotation in the scene.

Here are some useful settings of the sensor-interface component:

Setting:	*Description:*
Device Streaming Mode	Device streaming mode. It can be a connected camera, a recording or disabled.
Device Index	Index of the camera in the list of currently connected cameras.
Recording File	Full path to the recording file, if the streaming mode is PlayRecording.
Min Distance	Minimum distance in meters, used for creating the depth-related images.
Max Distance	Maximum distance in meters, used for creating the depth-related images.
Point Cloud Resolution	Resolution of the generated point-cloud textures (color or depth camera res).
Point Cloud Vertex Texture	Render texture, used for point-cloud vertex mapping. The texture resolution should match the depth or color image resolution.
Point Cloud Color Texture	Render texture, used for point-cloud color mapping. The texture resolution should match the depth or color image resolution.

Short Descriptions of the Components in KinectScripts-Folder

Short descriptions of the general-purpose components located in KinectScripts-folder can be found in the online documentation.

How to Utilize the CVE-related Scripts and Components in Your Project

1. Install the 'Inference Engine'-package. See the 'How to setup and run…'-section above.
2. Copy the 'KinectScripts'-folder from 'ComputerVisionExamples'-folder of this package to your project. This folder contains the needed components, scripts, filters, and interfaces.
3. Copy the 'Resources'-folder from the 'ComputerVisionExamples'-folder of this package to your project. This folder contains the needed shaders and other resources.
4. Wait until Unity imports, and compiles the newly detected packages, resources, and scripts.
5. Please do not share the 'DemoScenes'-folder in source form or as part of public repositories.

More Information, Support and Feedback

Web: https://rfilkov.com/2024/01/22/computer-vision-examples-for-unity/
Docs: https://github.com/rfilkov/cvedocs/wiki

Contact: http://rfilkov.com/about/#contact (please mention your invoice or order number)
Twitter: https://twitter.com/roumenf
LinkedIn: https://www.linkedin.com/in/rfilkov/